

2024 Sep Oppe 1 - Set 1

Section 3 - Problem 1

Student Marks Filter

Problem Statement & Criteria

Given a list of tuples (rollno:str, marks:int), write a function **get_roll_nos(data, criteria)** to filter student roll numbers based on:

- **'above_average'**: Marks $\geq$ average.
- **'below_average'**: Marks $<$ average.
- **'fail'**: Marks < 40 .
- **'toppers'**: Marks ≥ 90 .
- **None**: Return all roll numbers.
- **Any other value**: Return None.

Example & Function Details

```
data = [('101', 85), ('102', 75), ('103', 45), ('104', 95), ('105', 35)]

print(get_roll_nos(data, 'above_average')) #
Output: ['101', '102', '104']
print(get_roll_nos(data, 'fail'))          # Output:
['105']
print(get_roll_nos(data, 'toppers'))       # Output:
['104']
print(get_roll_nos(data))                  # Output:
['101', '102', '103', '104', '105']
```

CRITICAL: The solution must preserve the original **order** of the data. This is a function-only question; do not take input or print output within the final function definition.

The Challenge: Filtering Student Data

Input: List of Tuples

```
data = [  
    (101, 85),  
    (102, 35),  
    (103, 92),  
    (104, 45)  
]
```

→ Tuple Structure:
(Roll No, Marks)

Target Output

Function: `'get_roll_nos(data, criteria)'`

```
# Returns list of integers  
[101, 103, 104]
```

CRITICAL: The solution must preserve the original order of the data.

Defining the Filtering Criteria

Criteria Argument	Logic Rule	Description
'above_average'	Marks $\geq$ Average	Dynamically calculated based on class mean.
'below_average'	Marks $<$ Average	Dynamically calculated based on class mean.
'fail'	Marks $<$ 40	Static threshold for failing grade.
'toppers'	Marks $\geq$ 90	Static threshold for top performers.
None	Select All	Returns every roll number in the list.
Any other string	Invalid	Must return None immediately.

The Logical Strategy: A Two-Phase Approach



Phase 1: Preparation

Validate Inputs
(Check string validity)



Calculate Global Stats
(Class Average)



**Perform ONCE. Do not
calculate inside the loop.**



Pass "Average" Variable



Phase 2: Execution

Single Pass Iteration
($O(n)$)

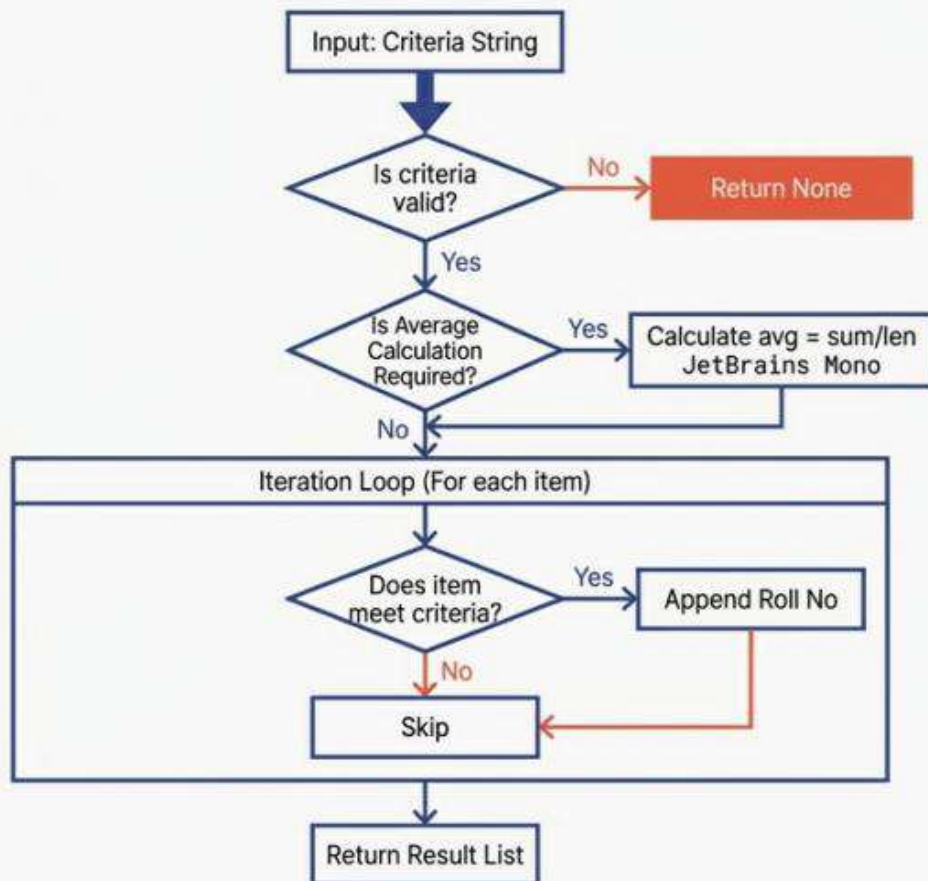


Conditional Logic Check



Append to Result List

The Decision Tree



Approach A: The Explicit Manual Loop

Readable, debuggable, and structurally clear.

Calculated once, outside the loop.

Tuple Unpacking: separating (r, m)
immediately for clarity.

Explicit branching for easy debugging.

```
def get_roll_nos(data, criteria=None):
    valid = ['above_average', 'below_average', 'fail', 'toppers', None]

    if criteria not in valid:
        return None

    # Safe average calculation
    avg = sum(m for r, m in data) / len(data) if data else 0

    res = []
    for roll, marks in data:
        if criteria == 'above_average' and marks >= avg:
            res.append(roll)
        elif criteria == 'fail' and marks < 40:
            res.append(roll)
        # ... (other conditions)
        elif criteria is None:
            res.append(roll)

    return res
```

Why the Explicit Approach Works

Readability

The `if/elif` block structure mirrors human decision-making. It physically separates the logic for failing grades from toppers, making the intention obvious to any reader.

Debug-Friendly

Because each condition has its own line, you can easily insert breakpoints or `print()` statements to isolate why a specific specific roll number is failing a check.

Input Safety

The initial validation block acts as a gatekeeper. By checking `if criteria not in valid` first, we prevent the function from processing garbage data.

Order Preservation

By iterating sequentially and appending to `res = []`, we guarantee the output list maintains the exact chronological order of the input list.

Approach B: The Elegant List Comprehension

Condensing logic without sacrificing architecture.

The Logic Core

```
return [r for r, m in data if  
    { (criteria is None) or  
      (criteria == 'above_average' and m >= avg) or  
      (criteria == 'below_average' and m < avg) or  
      (criteria == 'fail' and m < 40) or  
      (criteria == 'toppers' and m >= 90) ]
```

Mapping Logic to Comprehension

```
[r | for r, m in data | if (criteria == 'fail' and m < 40)...]
```

The Payload

Collect this specific value.

The Iterator

Walk through the list, unpacking tuple items.

The Filter

Only keep items where this boolean chain is True.

Flattened Logic

Vertical structure becomes a horizontal chain.

Comparative Analysis

Manual Loop

The Explicit Path

- **Pro:** ✓ Beginner friendly & Debuggable
- **Con:** ✗ Verbose boilerplate code
- **Best For:**
Complex logic requiring multiple intermediate steps or side effects.

List Comprehension

The Elegant Path

- **Pro:** ✓ Concise & Pythonic
- **Con:** ✗ Can become visually dense
- **Best For:**
Standard filtering and mapping tasks where logic fits in a boolean expression.

Shared Strength:
 $O(n)$ Efficiency & Stable Ordering



Engineering Robustness (Edge Cases)

Empty Data



Risk

ZeroDivisionError

Calculating average on an empty list crashes the program.

The Fix

```
avg = sum(...) / len(data) if data else 0
```

Handles empty data

Invalid Inputs



Risk

Silent Failures

Passing strings like 'best_students' might return an empty list instead of an error, confusing the user.

The Fix



```
if criteria not in valid:  
    return None
```

Validates inputs upfront

Key Takeaways & Best Practices

1 ✓ Validation First

Always sanitize inputs (strings, types) before beginning expensive calculations.

2 ✓ Optimize the Math

Calculate global statistics (like averages) *outside* your loops. Complexity should remain $O(n)$.

3 ✓ Preserve Order

Iterating through the source list sequentially is the safest way to maintain data integrity.

4 ✓ Know Your Tool

Use **Manual Loops** for debugging complex flows.
Use **Comprehensions** for clean, standard filtering.